INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SANTA CATARINA - CÂMPUS FLORIANÓPOLIS
DEPARTAMENTO ACADÊMICO DE ELETRÔNICA
CURSO DE GRADUAÇÃO EM ENGENHARIA ELETRÔNICA

JOSÉ HENRIQUE SCHMITT FIDELIS

SISTEMA DE SEGUIMENTO DE PESSOAS COM VISÃO COMPUTACIONAL EMBARCADA NO RASPBERRY PI ZERO 2 W

FLORIANÓPOLIS, 2026

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SANTA CATARINA - CÂMPUS FLORIANÓPOLIS
DEPARTAMENTO ACADÊMICO DE ELETRÔNICA
CURSO DE GRADUAÇÃO EM ENGENHARIA ELETRÔNICA

JOSÉ HENRIQUE SCHMITT FIDELIS

SISTEMA DE SEGUIMENTO DE PESSOAS COM VISÃO COMPUTACIONAL EMBARCADA NO RASPBERRY PI ZERO 2 W

Trabalho de Conclusão de Curso submetido ao Instituto Federal de Educação, Ciência e Tecnologia de Santa Catarina como parte dos requisitos para obtenção do título de Bacharel em Engenharia Eletrônica.

Orientador:
Prof. Matheus Leitzke Pinto, Me

FLORIANÓPOLIS, 2026

PÁGINA PARA COLOCAÇÃO
DA FICHA DE IDENTIFICAÇÃO DA OBRA

(A Ficha de identificação da obra deve ser elaborada de acordo com o padrão adotado pela biblioteca do IFSC a partir do formulário disponível em http://ficha.florianopolis.ifsc.edu.br/

Observação: por questões de compatibilidade do *site*, recomenda-se que a Ficha de identificação seja gerada no navegador ***Mozilla Firefox***)

TÍTULO DO TRABALHO

NOME DO AUTOR

Este trabalho foi julgado adequado para obtenção do título de Engenheiro/Tecnólogo/Especialista/Mestre em XXXX e aprovado na sua forma final pela banca examinadora do Curso XXXXXX do Instituto Federal de Educação, Ciência e Tecnologia de Santa Catarina.

Florianópolis, XX de XXXXXX, 20XX.

Banca Examinadora:

_____
Nome do Orientador, Titulação

_____
Nome do Coorientador (se houver), Titulação
Instituição/Empresa

_____
Nome do Membro da Banca, Titulação
Instituição/Empresa

_____
Nome do Membro da Banca, Titulação
Instituição/Empresa

(Dedicatória é um elemento opcional.
Texto alinhado no canto inferior direito.
Não deve ultrapassar uma página.)

AGRADECIMENTOS

Elemento opcional que não pode ultrapassar o limite de uma página.

(Epígrafe é um elemento opcional.
Texto alinhado no canto inferior direito.
Não deve ultrapassar uma página.)

RESUMO

O resumo deve mostrar a natureza e o objetivo do trabalho, o método que foi empregado, os resultados e as conclusões. O resumo deve conter entre 150 e 500 palavras e constitui-se de um único parágrafo, sem recuo.

Palavras-chave: Visão computacional. Robótica móvel. Seguimento de pessoas.

ABSTRACT

The abstract should show the nature and scope of work, the method that was used, the results and conclusions. The abstract may contain between 150 and 500 words, and it must be only one paragraph.

Keywords: Computer vision. Mobile robotics. Person following.

Lista de Figuras

LISTA DE QUADROS

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

BGR	Blue, Green and Red
CNN	Convolutional Neural Network
CPU	Unidade central de processamento
CSI-2	Camera Serial Interface 2
DAELN	Departamento Acadêmico de Eletrônica
IFSC	Instituto Federal de Santa Catarina
IHM	Interface Homem-Máquina
IoT	Internet das Coisas – *Internet of Things*
IoU	Intersection over Union
JPEG	Joint Photographic Experts Group
LPDDR2	Low Power Double Data Rate 2
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
RAM	Memória de acesso aleatório
RGB	Red, Green and Blue
ReLU	Rectified Linear Unit
SIMD	Single Instruction, Multiple Data
YOLO	You Only Look Once

SUMÁRIO

1 INTRODUÇÃO 14
1.1 Problema de pesquisa 16
1.2 Justificativa 16
1.3 Objetivo geral 17
1.4 Objetivos específicos 17
2 FUNDAMENTAÇÃO TEÓRICA 19
2.1 Imagens digitais e visão computacional 19
2.1.1 Representação de uma imagem 19
2.1.2 Da imagem numérica à informação sobre objetos 20
2.1.3 Preparação das imagens para a rede 20
2.2 Redes neurais convolucionais 21
2.2.1 Aprendizado a partir de exemplos 21
2.2.2 Pesos e funções de ativação 21
2.2.3 Convolução e mapas de características 22
2.2.4 Dimensões e composição das camadas 23
2.2.5 Generalização e modelos pré-treinados 23
2.3 Detecção de objetos e família YOLO 24
2.3.1 Caixas delimitadoras 24
2.3.2 Classe e confiança da previsão 24
2.3.3 Sobreposição e detecções redundantes 25
2.3.4 A proposta de YOLO 25
2.3.5 Componentes e variantes atuais 25
2.4 Inferência em sistemas embarcados 26
2.4.1 Do modelo treinado às operações no processador 26
2.4.2 Armazenamento e memória durante a execução 27
2.4.3 Resolução e custo da rede 27
2.4.4 Precisão numérica e quantização 28
2.4.5 Exportação e bibliotecas de inferência 28
2.4.6 Contexto do Raspberry Pi Zero 2 W 28
3 METODOLOGIA 30
3.1 Métodos Aplicados 30
4 APRESENTAÇÃO DOS RESULTADOS 31
4.1 Análise e discussão dos resultados 31
5 CONSIDERAÇÕES FINAIS 32
5.1 Sugestões para trabalhos futuros 32
REFERÊNCIAS 33
APÊNDICES 36
APÊNDICE A – TÍTULO 37
APÊNDICE B – TÍTULO 38
ANEXOS 39
ANEXO A – TÍTULO 40
ANEXO B – TÍTULO 41

1 INTRODUÇÃO

O desenvolvimento de técnicas de inteligência artificial e visão computacional tem ampliado as possibilidades de aplicação de sistemas capazes de interpretar informações obtidas por câmeras e utilizá-las para interagir com o ambiente. Entre essas aplicações encontram-se os sistemas robóticos móveis, nos quais a percepção visual pode ser empregada para detectar objetos e pessoas e fornecer informações utilizadas na tomada de decisões de movimento.

Uma das tarefas fundamentais nesse contexto é a detecção de objetos em imagens. Modelos baseados em redes neurais convolucionais possibilitaram avanços nesse campo, com destaque para a família You Only Look Once (YOLO), cuja proposta original consiste em realizar a predição de caixas delimitadoras e classes de objetos em uma única avaliação da rede (REDMON *et al.*, 2016). A existência de variantes com diferentes dimensões e requisitos computacionais permite investigar sua aplicação em dispositivos embarcados com recursos limitados (Ultralytics, s.d.c).

A execução de modelos de visão computacional diretamente no dispositivo responsável pela aquisição dos dados dispensa o envio contínuo das imagens para servidores externos e permite realizar a percepção visual sem depender de uma conexão com a Internet. Entretanto, essa abordagem exige adequar o processamento à capacidade computacional e à memória disponíveis, considerando também o tempo necessário para produzir informações úteis ao controle do robô.

Neste trabalho, será utilizado o Raspberry Pi Zero 2 W, um computador de placa única equipado com processador ARM Cortex-A53 de quatro núcleos operando a 1 GHz e 512 MB de memória de acesso aleatório (RAM) (Raspberry Pi Ltd, 2024). Essas características tornam a plataforma de interesse para aplicações embarcadas de baixo custo, ao mesmo tempo que impõem restrições à execução de modelos de aprendizado profundo. A escolha do modelo, da resolução de entrada e do formato de execução será considerada na adequação do sistema a essas restrições.

Nesse contexto, este trabalho propõe o desenvolvimento e a avaliação de um sistema de seguimento de pessoas com processamento de visão computacional realizado integralmente em um Raspberry Pi Zero 2 W. Uma Raspberry Pi Camera, modelo [A preencher: modelo da câmera], será utilizada para adquirir imagens do ambiente, que serão processadas por modelos pré-treinados da família YOLO. As detecções serão filtradas para a classe correspondente a pessoas, e as posições e dimensões das caixas delimitadoras fornecerão as informações visuais utilizadas no acompanhamento.

A seleção inicial do alvo considerará a primeira pessoa detectada. Caso sejam detectadas várias pessoas na mesma imagem durante essa seleção, uma delas

será escolhida aleatoriamente. Nas imagens seguintes, será empregado um algoritmo simples de associação espacial entre as detecções atuais e a posição anteriormente observada do alvo. Essa estratégia buscará manter a continuidade do acompanhamento, mas estará sujeita a trocas de alvo em situações como cruzamentos entre pessoas, oclusões ou perdas prolongadas de detecção, uma vez que não será baseada no reconhecimento da identidade dos indivíduos.

As informações do alvo serão utilizadas por um algoritmo de controle de alto nível para determinar referências de velocidade linear e angular para a plataforma robótica móvel. O desvio horizontal do centro da caixa delimitadora em relação ao centro da imagem será empregado para orientar a rotação do veículo. Para o movimento de aproximação ou afastamento, será investigado o uso das dimensões da caixa delimitadora como indicador indireto de proximidade.

Será avaliada uma calibração básica que relacione o tamanho aparente da pessoa na imagem a distâncias conhecidas, com o propósito de estabelecer uma distância aproximada de referência para o acompanhamento. A precisão dessa estimativa será verificada experimentalmente, considerando suas limitações diante de variações na altura, na postura e na visibilidade da pessoa. Dessa forma, a manutenção da distância será tratada como um resultado a ser avaliado, sem pressupor a obtenção de uma medição precisa apenas a partir da caixa delimitadora.

Quando o alvo deixar de ser detectado, o sistema interromperá o deslocamento linear e realizará uma busca por rotação lenta, orientada pela tendência de movimento horizontal observada nas últimas detecções válidas. Os critérios para caracterizar a perda do alvo, limitar a duração da busca e tratar situações sem uma tendência de movimento identificável serão definidos na metodologia. A recuperação do acompanhamento utilizará as informações visuais disponíveis, estando sujeita às limitações da associação espacial.

Os comandos produzidos pelo Raspberry Pi serão enviados a um microcontrolador ESP32 previamente integrado à plataforma robótica e responsável pela execução dos comandos de movimento. Dessa forma, o sistema será dividido entre uma camada de percepção e decisão, executada no Raspberry Pi, e uma camada de controle de movimento, executada pelo microcontrolador. O projeto concentra-se na integração entre a visão computacional embarcada e uma plataforma móvel existente, não abrangendo o desenvolvimento dos circuitos de acionamento dos motores.

Diferentes modelos e resoluções de entrada serão avaliados experimentalmente, iniciando-se por configurações de menor custo computacional e ampliando-se os testes conforme a capacidade do hardware. Essa abordagem permitirá investigar o compromisso entre qualidade das detecções e tempo de processamento. A seleção das configurações e as condições de comparação serão descritas na metodologia,

considerando o desempenho do sistema durante a execução conjunta da aquisição de imagens, da detecção e do controle de alto nível.

A avaliação considerará o tempo de inferência, a taxa efetiva de processamento de imagens e o tempo decorrido entre a aquisição de uma imagem e o envio do comando de movimento correspondente ao ESP32. A qualidade das detecções será analisada por meio de acertos, falsas detecções e falhas de detecção em amostras de imagens com referência estabelecida por avaliação manual. Durante o seguimento, serão observados o erro horizontal normalizado pela largura da imagem, a continuidade do acompanhamento, as perdas e eventuais trocas de alvo, além do comportamento da estimativa de distância em comparação com medidas de referência obtidas nos ensaios.

Os experimentos serão realizados em ambiente predominantemente interno, com condições adequadas de iluminação, área de circulação livre de obstáculos e velocidades limitadas. O desvio de obstáculos ficará fora do escopo do trabalho. As condições dos ensaios, incluindo as velocidades da plataforma e da pessoa, a faixa de distâncias e os critérios de sucesso do acompanhamento, serão estabelecidas na metodologia: [A preencher: condições experimentais e critérios quantitativos de avaliação].

Dessa forma, o trabalho busca investigar se e em quais condições as configurações avaliadas permitem realizar o seguimento de uma pessoa por meio de visão computacional executada localmente no Raspberry Pi Zero 2 W. A análise relacionará o desempenho das detecções e os atrasos de processamento ao comportamento da plataforma, permitindo caracterizar as possibilidades e limitações da solução nas condições experimentais adotadas.

1.1 Problema de pesquisa

Em quais condições de operação e com quais configurações de modelo YOLO e resolução de entrada, dentre as avaliadas, é possível realizar o seguimento de uma pessoa por uma plataforma robótica móvel de baixa velocidade utilizando processamento de visão computacional integralmente local no Raspberry Pi Zero 2 W, considerando a qualidade das detecções, o tempo de resposta e o desempenho do acompanhamento?

1.2 Justificativa

O seguimento visual de pessoas constitui uma aplicação que integra percepção, processamento e controle em uma tarefa física, permitindo avaliar como as informações extraídas de imagens podem orientar o movimento de uma plataforma robótica. Sua implementação em um dispositivo com recursos computacionais limi-

tados oferece um contexto experimental para estudar a adequação de modelos de inteligência artificial às exigências de uma aplicação embarcada.

Embora o desempenho de um detector possa ser caracterizado isoladamente, sua utilização em uma plataforma móvel depende também do tempo necessário para adquirir imagens, interpretar as detecções e produzir comandos de movimento. Assim, a avaliação do sistema integrado permite investigar como a qualidade e a frequência das informações visuais se relacionam com a capacidade de acompanhar uma pessoa. [A preencher: referências sobre seguimento visual de pessoas e avaliação de visão computacional embarcada].

A utilização do Raspberry Pi Zero 2 W e de uma plataforma existente controlada por ESP32 permite concentrar o estudo na execução local dos modelos e na integração entre percepção e movimento. O trabalho poderá contribuir com uma caracterização experimental das configurações avaliadas, das condições em que o acompanhamento se mostra viável e das limitações observadas no hardware utilizado.

1.3 Objetivo geral

Desenvolver e avaliar um sistema embarcado de visão computacional para seguimento de pessoas, utilizando modelos da família YOLO executados localmente em um Raspberry Pi Zero 2 W e integrados a uma plataforma robótica móvel controlada por ESP32, investigando a relação entre qualidade das detecções, tempo de resposta e capacidade de acompanhamento do alvo em condições controladas de operação.

1.4 Objetivos específicos

a) Implementar a aquisição e o processamento das imagens provenientes de uma Raspberry Pi Camera no Raspberry Pi Zero 2 W.

b) Implementar a detecção de pessoas utilizando modelos pré-treinados da família YOLO, restringindo as detecções utilizadas pelo sistema à classe correspondente a pessoas.

c) Desenvolver uma estratégia de seleção inicial do alvo e de manutenção do acompanhamento por associação espacial entre detecções sucessivas.

d) Avaliar diferentes modelos e resoluções de entrada, considerando a qualidade das detecções, o tempo de inferência e a taxa efetiva de processamento de imagens.

e) Investigar uma calibração básica entre o tamanho aparente da pessoa e distâncias conhecidas, avaliando sua utilização como referência aproximada para o controle da distância de acompanhamento.

f) Desenvolver um algoritmo de controle de alto nível que utilize a posição e as dimensões da caixa delimitadora do alvo para produzir referências de velocidade linear e angular.

g) Integrar o sistema executado no Raspberry Pi Zero 2 W ao ESP32 responsável pelo controle da plataforma móvel.

h) Implementar uma estratégia de busca do alvo por rotação lenta, orientada pela tendência de movimento observada nas últimas detecções válidas.

i) Medir o tempo entre a aquisição da imagem e o envio do comando de movimento ao ESP32, relacionando-o ao desempenho do acompanhamento.

j) Avaliar experimentalmente a centralização do alvo na imagem, a continuidade do seguimento, a recuperação após perdas de detecção e o comportamento do controle de distância em área livre de obstáculos.

k) Analisar a viabilidade e as limitações das configurações avaliadas para a execução integralmente local do sistema de visão computacional no Raspberry Pi Zero 2 W.

2 FUNDAMENTAÇÃO TEÓRICA

Este texto apresenta os fundamentos necessários para compreender a detecção de pessoas por redes neurais executadas em um sistema embarcado. A exposição parte da representação digital das imagens, introduz o funcionamento das redes convolucionais, apresenta a detecção de objetos com YOLO e discute a execução de modelos em hardware com recursos limitados. As escolhas de implementação, o acompanhamento do alvo, o controle da plataforma e os procedimentos experimentais serão tratados posteriormente, na metodologia e no desenvolvimento.

Os exemplos numéricos apresentados neste capítulo são didáticos e não representam resultados medidos no projeto.

2.1 Imagens digitais e visão computacional

2.1.1 Representação de uma imagem

Para o processamento computacional, uma imagem digital pode ser representada por um arranjo de valores numéricos. Sua organização espacial é dada por linhas e colunas, cujas posições correspondem aos pixels. Em uma representação colorida Red, Green and Blue (RGB), cada posição contém três componentes, associadas ao vermelho, ao verde e ao azul. Quando cada componente utiliza oito bits sem sinal, seus valores variam de 0 a 255. Essa é uma representação comum, embora existam imagens com outros números de canais e outras profundidades numéricas (Stanford University, s.d.b).

Considere, como exemplo, uma imagem RGB com largura de 640 pixels e altura de 480 pixels. Ela contém 307.200 posições espaciais e 921.600 valores de componentes de cor. Se cada valor ocupar um byte, o arranjo exige 921.600 bytes, aproximadamente 0,88 MiB, antes de considerar metadados, alinhamento ou cópias. Aqui, um mebibyte, ou MiB, corresponde a 2^{20} bytes. Esse cálculo descreve o arranjo descomprimido; não corresponde necessariamente ao tamanho de um arquivo Joint Photographic Experts Group (JPEG) salvo em disco.

A organização dos dados também faz parte da representação. Um tensor é, neste contexto, um arranjo numérico com uma ou mais dimensões. Uma imagem pode ser organizada como altura $\times$ largura $\times$ canais, enquanto bibliotecas de redes neurais podem utilizar lote $\times$ canais $\times$ altura $\times$ largura. A ordem dos canais também precisa ser conhecida: RGB e Blue, Green and Red (BGR) contêm as mesmas componentes, mas em posições diferentes. A documentação de inferência da Ultralytics, por exemplo, especifica convenções distintas para imagens NumPy e tensores de entrada (Ultralytics, s.d.a).

2.1.2 Da imagem numérica à informação sobre objetos

As tarefas de visão computacional se distinguem pela informação que produzem. Na classificação, o resultado indica uma categoria ou a presença de determinadas categorias na imagem. Na detecção, também é necessário localizar os objetos, normalmente por caixas delimitadoras. Na segmentação, a descrição espacial é mais detalhada, atribuindo rótulos a regiões ou pixels. Essas diferenças aparecem, por exemplo, na definição das tarefas do desafio PASCAL VOC (PASCAL VOC, 2010).

Para situar essas tarefas, imagine uma imagem com duas pessoas. Uma classificação pode indicar a presença da categoria “pessoa”. Um detector pode retornar duas caixas, cada uma associada à categoria prevista. Uma segmentação pode representar as regiões ocupadas pelos corpos. O resultado de detecção, isoladamente, fornece categoria e localização. Neste texto, “detecção de pessoas” designa essa tarefa; a correspondência entre instantes não integra a saída aqui definida (PASCAL VOC, 2010).

A relação entre os valores dos pixels e a categoria de um objeto não é fixa. Iluminação, escala, orientação, postura, fundo e oclusões podem alterar a aparência observada. Pessoas diferentes também apresentam variações de roupa e características visuais. Portanto, o sistema precisa lidar com variações dentro da mesma categoria, além de distinguir categorias diferentes (Stanford University, s.d.b).

2.1.3 Preparação das imagens para a rede

O pré-processamento adapta a imagem ao formato esperado pelo modelo. Uma operação frequente é o redimensionamento, que calcula uma nova grade de pixels a partir da imagem original. Para isso, utilizam-se métodos de interpolação, como vizinho mais próximo, bilinear ou métodos baseados em áreas. Redimensionar para uma quantidade menor de pixels reduz a representação espacial disponível; ampliar uma imagem por interpolação não equivale a adquirir novos detalhes com a câmera (OpenCV, s.d.).

As proporções também precisam ser consideradas. Em um exemplo geométrico, transformar diretamente 640×480 em 320×320 aplica fatores diferentes aos dois eixos. Uma alternativa é redimensionar proporcionalmente e completar as bordas, procedimento conhecido como *letterbox*. O preenchimento e as dimensões efetivas dependem da implementação e da configuração do detector. Ao interpretar as caixas previstas, deve-se considerar a transformação realizada sobre a imagem (Ultralytics, s.d.a).

Outra operação é a adequação da escala numérica dos valores. Dependendo da rede, isso pode envolver divisão por uma constante, subtração de médias ou divisão por desvios-padrão. A transformação deve seguir a convenção prevista pelo

modelo: normalizar valores e redimensionar a imagem são operações distintas. A primeira altera a representação numérica; a segunda modifica as dimensões espaciais. A preparação utilizada na inferência deve ser compatível com aquela associada ao treinamento (Stanford University, s.d.d).

2.2 Redes neurais convolucionais

2.2.1 Aprendizado a partir de exemplos

O aprendizado de máquina permite ajustar modelos a partir de dados. No aprendizado supervisionado, os exemplos são acompanhados por respostas de referência, como classes de imagens ou anotações de objetos. Uma rede neural aplica transformações parametrizadas à entrada e produz uma saída. Durante o treinamento, seus parâmetros são ajustados para reduzir uma função de perda, que expressa a discrepância entre as previsões e as referências segundo o problema considerado (LECUN; BENGIO; HINTON, 2015).

O treinamento inclui a propagação dos dados pela rede, o cálculo da perda e a obtenção de gradientes por retropropagação. Um algoritmo de otimização utiliza esses gradientes para atualizar os parâmetros. Na inferência, a rede utiliza os parâmetros já aprendidos para calcular respostas para novas entradas. Assim, empregar um modelo pré-treinado na câmera não significa que ele esteja aprendendo continuamente com as imagens recebidas (LECUN; BENGIO; HINTON, 2015).

2.2.2 Pesos e funções de ativação

Uma unidade básica de uma rede pode ser descrita por uma soma ponderada seguida de uma função de ativação:

$$\begin{aligned} z &= \sum_{i=1}^{n} w_i x_i + b, \\ y &= f(z). \end{aligned} \tag{1}$$

Na Equação 1, x_i são valores de entrada, w_i são pesos, b é um termo de deslocamento, chamado viés, e f é a função de ativação. Os pesos determinam a contribuição das entradas. A composição de unidades e camadas permite produzir transformações mais elaboradas. Funções não lineares são necessárias para que uma sequência de camadas não se reduza apenas a uma transformação afim equivalente. Um exemplo didático é Rectified Linear Unit (ReLU), definida por $f(z) = \max(0, z)$ (Stanford University, s.d.c).

Por exemplo, uma unidade com entradas 0,2 e 0,8, pesos 0,5 e -0,25 e viés 0,1 produz $z = 0{,}5 \cdot 0{,}2 - 0{,}25 \cdot 0{,}8 + 0{,}1 = 0$. Alterar os pesos modifica a resposta

à mesma entrada. O exemplo ilustra o papel dos parâmetros, sem representar um neurônio específico do modelo utilizado.

2.2.3 Convolução e mapas de características

As redes neurais convolucionais, do inglês Convolutional Neural Networks (CNNs), exploram a estrutura espacial dos dados. Em uma camada convolucional, cada saída depende de uma região local da entrada, chamada campo receptivo local. Um mesmo conjunto de pesos é aplicado a diferentes posições, produzindo um mapa de características. Esse compartilhamento permite procurar uma característica em vários locais sem aprender um conjunto independente de pesos para cada posição. O trabalho de LeCun et al. sobre reconhecimento de documentos apresenta esses princípios em uma aplicação clássica de redes convolucionais (LECUN *et al.*, 1998).

O conjunto de pesos aplicado à região local é chamado filtro ou *kernel*. A operação envolve multiplicar valores correspondentes da entrada e do filtro e somar os produtos. Nas implementações usuais de CNNs, essa operação geralmente corresponde à correlação cruzada, na qual não se inverte espacialmente o filtro. A expressão "convolução" permanece consagrada para a camada na literatura de aprendizado profundo (DUMOULIN; VISIN, 2016).

Um exemplo com um único canal ajuda a visualizar a operação, conforme a Equação 2:

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \tag{2}$$
$$y = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 + 4 \cdot (-1) = -3.$$

Ao deslocar esse filtro, calculam-se resultados para outras regiões. Em uma convolução convencional, sem agrupamento, o filtro considera todos os canais da entrada e acumula as contribuições correspondentes. Diferentes filtros produzem diferentes canais de saída. Seus valores são aprendidos no treinamento, permitindo extrair características úteis ao objetivo da rede (Stanford University, s.d.a).

O compartilhamento pode ser ilustrado por uma camada convolucional convencional, sem agrupamento, com 16 filtros de tamanho 3×3 sobre uma entrada de três canais. Considerando um viés por filtro, existem $16 \cdot (3 \cdot 3 \cdot 3 + 1) = 448$ parâmetros. Esse número não depende diretamente de quantas posições da imagem serão percorridas. Entretanto, o número de operações aumenta quando há mais posições de saída. O exemplo separa duas grandezas: quantidade de parâmetros e quantidade de cálculos (LECUN *et al.*, 1998).

2.2.4 Dimensões e composição das camadas

O passo de deslocamento do filtro é denominado *stride*. Um passo maior pode produzir um mapa espacialmente menor. O *padding* acrescenta posições nas bordas, geralmente preenchidas com zeros, e influencia o tamanho da saída e o tratamento dos pixels próximos aos limites. Para uma dimensão de entrada i, filtro de tamanho k, preenchimento simétrico p e passo s, sem dilatação, a dimensão de saída é $\left\lfloor \frac{i+2p-k}{s} \right\rfloor + 1$. Por exemplo, entrada 5, filtro 3, preenchimento 0 e passo 1 resultam em saída 3 (DUMOULIN; VISIN, 2016).

As CNNs também podem empregar operações de redução, como *pooling*, que agregam valores de uma vizinhança por uma regra, por exemplo máximo ou média. Camadas de ativação introduzem não linearidades entre transformações. A combinação concreta desses elementos depende da arquitetura; a presença de convoluções não obriga toda CNN a ter a mesma sequência de camadas ou a utilizar pooling (Stanford University, s.d.a).

Ao encadear camadas que combinam posições espaciais, o campo receptivo em relação à imagem original pode aumentar. A rede pode, assim, combinar informações locais em representações mais abrangentes. Operações pontuais, como uma ativação aplicada separadamente a cada posição, não ampliam por si só essa região. O significado das representações depende dos parâmetros aprendidos, sem exigir que cada filtro corresponda a uma parte fixa do objeto (ZHANG *et al.*, 2023, seção 7.2).

2.2.5 Generalização e modelos pré-treinados

A finalidade do treinamento é obter um modelo que produza respostas úteis também para exemplos não utilizados no ajuste. Essa capacidade é denominada generalização. Um modelo pode apresentar baixo erro nos dados de treinamento e ainda responder inadequadamente a novas imagens, fenômeno associado ao sobreajuste. Por isso, desempenho no conjunto de treinamento não constitui, por si só, garantia de funcionamento em outras condições (GOODFELLOW; BENGIO; COURVILLE, 2016, seção 5.2).

Modelos pré-treinados permitem aproveitar parâmetros obtidos em um treinamento anterior. É possível utilizá-los diretamente para inferência ou realizar um novo ajuste com dados específicos, procedimento denominado ajuste fino, ou *fine-tuning*. Essas alternativas são distintas: carregar pesos e processar imagens não executa automaticamente um novo treinamento. A adequação dos pesos ao problema depende da relação entre o domínio original e as imagens de interesse (ZHANG *et al.*, 2023, seção 14.2).

2.3 Detecção de objetos e família YOLO

2.3.1 Caixas delimitadoras

Um detector de objetos retorna previsões de localização e categoria. A localização pode ser representada por uma caixa delimitadora retangular alinhada aos eixos da imagem. Na convenção `xyxy`, os limites são $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$. Na convenção `xywh` empregada pela Ultralytics, a representação utiliza centro, largura e altura. Também existem coordenadas normalizadas em relação às dimensões da imagem (Ultralytics, s.d.b).

Adotando coordenadas contínuas no plano da imagem, as relações geométricas são dadas pela Equação 3:

$$\begin{aligned} c_x &= \frac{x_{\min} + x_{\max}}{2}, \quad c_y = \frac{y_{\min} + y_{\max}}{2}, \\ w &= x_{\max} - x_{\min}, \quad h = y_{\max} - y_{\min}. \end{aligned} \tag{3}$$

Por exemplo, a caixa $(100, 40, 220, 360)$ tem centro $(160, 200)$, largura 120 e altura 320, nas unidades de coordenadas adotadas. Essas informações descrevem a extensão prevista do objeto na imagem. A caixa pode incluir partes do fundo, pois não representa seu contorno exato. A distinção entre localização por caixa e descrição precisa da forma é parte da definição da tarefa de detecção (Ultralytics, s.d.b).

2.3.2 Classe e confiança da previsão

Além da caixa, o detector produz informações sobre a categoria prevista e uma pontuação de confiança. Essa pontuação é utilizada para ordenar previsões e aplicar limiares. Um limiar mais alto admite um subconjunto mais restrito de previsões, mas a decisão sobre quais detecções são úteis depende do problema. A interpretação do score como probabilidade de acerto exige considerar a calibração do detector: uma pontuação 0,8 não deve ser automaticamente apresentada como uma garantia de 80% de acertos em qualquer situação (KÜPPERS *et al.*, 2020).

É útil distinguir a decisão da rede e a decisão de uso da aplicação. A rede produz previsões; a aplicação pode conservar somente algumas classes ou pontuações. No código da Ultralytics consultado, o filtro de classes atua sobre previsões já produzidas. Consequentemente, selecionar apenas a classe “pessoa” não transforma automaticamente uma rede multiclasse em uma rede menor, embora possa reduzir o trabalho posterior sobre as saídas. Essa conclusão decorre do fluxo da implementação consultada, não de uma medição de ganho de desempenho neste projeto (Ultralytics, 2026).

2.3.3 Sobreposição e detecções redundantes

A interseção sobre união, do inglês Intersection over Union (IoU), expressa a sobreposição entre duas caixas:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{4}$$

Na Equação 4, $|A \cap B|$ representa a área de interseção e $|A \cup B|$ a área de união. Para caixas válidas e união de área positiva, o resultado varia de zero, quando não há área comum, a um, quando as caixas coincidem. Se duas caixas têm área 100 cada e interseção de área 50, sua união tem área 150 e a IoU é $50/150$, aproximadamente 0,33. É uma relação geométrica entre caixas, distinta da pontuação de confiança produzida pela rede (REZATOFIGHI *et al.*, 2019, seção 3).

Alguns detectores podem gerar várias caixas para um mesmo objeto. Na supressão de não máximos, ou Non-Maximum Suppression (NMS), previsões são ordenadas por pontuação; uma caixa selecionada é mantida e caixas suficientemente sobrepostas podem ser descartadas segundo um limiar. O tratamento pode ser realizado por classe. A finalidade é reduzir redundâncias, mas a necessidade dessa etapa depende da arquitetura e do caminho de inferência (Ultralytics, 2026).

2.3.4 A proposta de YOLO

YOLO é a sigla de *You Only Look Once*. Na proposta original de Redmon et al., a detecção é formulada como uma predição conjunta de caixas e classes a partir da imagem completa em uma avaliação da rede. Essa formulação caracteriza uma abordagem de um estágio: não há um módulo separado de propostas de regiões que depois sejam classificadas individualmente. O termo não significa uma rede de uma única camada ou ausência de preparação e tratamento das saídas (REDMON *et al.*, 2016).

A versão original organiza previsões por uma grade espacial e utiliza uma formulação específica para caixas, classes e confiança. Esses detalhes pertencem àquele modelo. A família evoluiu, e explicar uma versão posterior exige consultar sua arquitetura e implementação correspondentes. Assim, a ideia de detectar a partir de uma passagem pela rede pode ser apresentada como fundamento, enquanto números de células, quantidade de caixas por célula e fórmulas de score precisam ser associados à versão que os utiliza (REDMON *et al.*, 2016).

2.3.5 Componentes e variantes atuais

Uma organização conceitual útil distingue extração de características (*backbone*), combinação de características (*neck*) e produção das previsões (*head*). No

YOLO26, os autores descrevem componentes de extração e fusão de características e cabeças destinadas às tarefas de saída. Para a detecção, essas estruturas devem produzir informações de localização e categoria. Diferentes escalas do modelo, identificadas como `n`, `s`, `m`, `l` e `x`, correspondem a configurações distintas da família. O artigo que descreve YOLO26 é, na versão consultada, um preprint (JOCHER *et al.*, 2026).

YOLO26 oferece caminhos de detecção com associação de treinamento *one-to-many* e *one-to-one*: associam-se múltiplas previsões a um objeto, no primeiro caso, ou uma previsão a cada objeto, no segundo. O primeiro caminho utiliza NMS; o segundo permite inferência sem essa supressão. Portanto, NMS não é uma etapa obrigatória de toda execução da família. O caminho efetivo depende da versão e da configuração, inclusive na exportação (JOCHER *et al.*, 2026; Ultralytics, s.d.c).

O artigo original e a documentação da versão adotada cumprem, portanto, papéis complementares: o primeiro fundamenta a proposta de detecção, e os demais descrevem a concretização atual. As características específicas do modelo selecionado, de seu arquivo exportado e de seus parâmetros de execução serão registradas na metodologia, permitindo relacionar a fundamentação ao experimento efetivamente realizado.

2.4 Inferência em sistemas embarcados

2.4.1 Do modelo treinado às operações no processador

A inferência de uma rede envolve executar operações numéricas sobre os dados de entrada e os parâmetros aprendidos. Nas convoluções, são realizadas muitas multiplicações e acumulações; outras camadas aplicam ativações, reorganizam dados ou combinam resultados. A execução também exige movimentar pesos e resultados intermediários entre níveis de memória. Portanto, o custo depende tanto do cálculo quanto do acesso aos dados, e não apenas da quantidade de camadas descritas no modelo (SZE *et al.*, 2017, seções III e V).

Essa execução pode ser representada como um grafo: os nós correspondem a operações, e as conexões indicam os dados consumidos e produzidos. O Open Neural Network Exchange (ONNX), por exemplo, define entradas, saídas, operadores, atributos e inicializadores, que podem armazenar pesos. Um runtime executa esse grafo no ambiente de destino (ONNX, s.d.).

Uma biblioteca de inferência permite integrar essas operações à aplicação embarcada. O NCNN é um exemplo, com implementação em C++ e interfaces para executar redes localmente em diferentes plataformas (Tencent, s.d.).

Na unidade central de processamento (CPU), o cálculo é realizado por instruções de máquina. A vetorização permite aplicar uma instrução a vários elementos,

abordagem denominada Single Instruction, Multiple Data (SIMD). A tecnologia Arm Neon oferece esse tipo de operação para dados inteiros e de ponto flutuante. Como exemplo conceitual, vários produtos entre entradas e pesos podem ser organizados em vetores, em vez de tratar cada par isoladamente. Essa forma de paralelismo ocorre no processamento das instruções e é distinta de simplesmente distribuir tarefas entre vários núcleos (Arm, s.d.).

2.4.2 Armazenamento e memória durante a execução

O arquivo de um modelo e a memória usada ao executá-lo representam grandezas diferentes. Além dos pesos, a execução mantém entradas, saídas e resultados intermediários, também chamados ativações, e pode utilizar áreas temporárias de trabalho. A organização e a reutilização dessas áreas dependem do runtime. Um grafo pode, por exemplo, permitir a reutilização de um buffer quando seu conteúdo anterior não é mais necessário (SZE *et al.*, 2017, seções V e VIII); (ONNX, s.d.).

Um cálculo simplificado ajuda a dimensionar os pesos. Se houver N parâmetros armazenados com b bits por parâmetro, seu armazenamento bruto será $N \cdot b/8$ bytes. Em um exemplo hipotético de dois milhões de parâmetros, 32 bits por valor correspondem a oito milhões de bytes, aproximadamente 7,63 MiB. Com 16 bits, seriam aproximadamente 3,81 MiB. O cálculo não inclui estruturas do arquivo, metadados, ativação, cópias ou outras partes do processo.

O mesmo raciocínio se aplica a um tensor. Uma entrada de 320×320 com três canais tem 307.200 elementos. A oito bits por elemento, são aproximadamente 0,29 MiB; a 32 bits, aproximadamente 1,17 MiB. Os exemplos mostram por que a profundidade numérica importa mesmo quando as dimensões espaciais não mudam. A representação efetiva deve ser aquela exigida pela rede e pela implementação, e não uma redução arbitrária dos dados.

2.4.3 Resolução e custo da rede

Reduzir a resolução da entrada e a largura de uma rede são estratégias distintas para diminuir seu custo. Em MobileNets, Howard et al. apresentam multiplicadores de resolução e de largura para explorar compromissos entre custo computacional e qualidade das respostas. A ideia geral é que mapas menores e menos canais reduzem a quantidade de operações e dados envolvidos, embora alterem a capacidade de representação disponível (HOWARD *et al.*, 2017).

Se largura e altura de um mapa forem reduzidas pela metade, o número de posições espaciais cai para um quarto. Mantidos filtro e canais de uma convolução convencional, essa redução diminui proporcionalmente o número de posições em que os produtos e somas são calculados. Ela não implica que a aplicação inteira execute

exatamente quatro vezes mais rápido: preparação de imagens, transferências, demais operações e custos fixos também participam da execução. Esse exemplo expressa uma relação de contagem, não um benchmark do Raspberry (HOWARD *et al.*, 2017).

2.4.4 Precisão numérica e quantização

A quantização representa valores utilizando um conjunto mais limitado de níveis numéricos. Uma aproximação comum relaciona um inteiro q a um valor real por uma escala s e um ponto zero z, na forma $r \approx s \cdot (q - z)$. O trabalho de Jacob et al. apresenta uma abordagem de treinamento e inferência que permite realizar operações com aritmética inteira, buscando reduzir custos em dispositivos de aplicação prática (JACOB *et al.*, 2018).

A redução de precisão pode diminuir armazenamento e custo de cálculo, mas introduz aproximações. O resultado depende do modelo, do método de quantização e do suporte do hardware e do software. Usar armazenamento de 16 bits, por exemplo, não significa necessariamente que todas as operações serão executadas nessa precisão ou que qualquer processador obterá aceleração. Assim, uma possibilidade de otimização precisa ser distinguida de uma característica realmente ativada e verificada na aplicação (JACOB *et al.*, 2018; Tencent, s.d.).

2.4.5 Exportação e bibliotecas de inferência

Exportar um modelo consiste em representá-lo para execução no ambiente de destino, sem necessariamente treiná-lo novamente. A compatibilidade envolve tipos de dados, dimensões e operações. No ONNX, os operadores possuem definições e versões que precisam ser suportadas pelo runtime (ONNX, s.d.).

O NCNN disponibiliza execução em CPU, otimizações para arquiteturas ARM e caminhos de aceleração para plataformas compatíveis. Seu ecossistema também oferece ferramentas de conversão de modelos. A existência desses recursos na biblioteca não garante que todos estejam disponíveis em uma placa particular. A aplicação que envolve o runtime pode adicionar dependências e consumo de recursos próprios. Por isso, distinguir rede, formato exportado, runtime e aplicação completa é necessário para compreender o que será instalado e executado (Tencent, s.d.).

2.4.6 Contexto do Raspberry Pi Zero 2 W

O Raspberry Pi Zero 2 W é um computador de placa única que utiliza processador Arm Cortex-A53 de quatro núcleos, arquitetura de 64 bits e frequência nominal de 1 GHz, acompanhado de 512 MB de memória Low Power Double Data Rate 2 (LPDDR2). A placa possui interface Camera Serial Interface 2 (CSI-2) para câmera e

armazenamento por cartão microSD. Essas características são apresentadas na documentação técnica do fabricante (Raspberry Pi Ltd, 2024, p. 2–3).

O tempo e a memória necessários dependem da combinação de rede, representação dos dados, runtime e plataforma. Resultados de outro processador não estabelecem o desempenho deste dispositivo; a análise precisa considerar a execução concreta do conjunto (SZE *et al.*, 2017, seções V e VIII).

A fundamentação permite compreender essa combinação sem antecipar os resultados do projeto. A metodologia poderá especificar o modelo e a configuração adotados. O desenvolvimento mostrará a integração da aquisição de imagens, do pré-processamento e da inferência no Raspberry.

3 METODOLOGIA

Texto texto texto

4 Apresentação dos Resultados

Texto texto texto

5 Considerações Finais

Texto texto texto

REFERÊNCIAS

Arm. *Neon*. s.d. Documentação técnica. Disponível em: https://www.arm.com/technologies/neon. Acesso em: 26 out. 2026. 27

DUMOULIN, V.; VISIN, F. *A guide to convolution arithmetic for deep learning*. 2016. ArXiv:1603.07285. Preprint. Versão 2, revisada em 11 jan. 2018. Disponível em: https://arxiv.org/html/1603.07285v2. Acesso em: 26 out. 2026. 22, 23

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016. Disponível em: https://www.deeplearningbook.org/. Acesso em: 26 out. 2026. 23

HOWARD, A. G. *et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. 2017. ArXiv:1704.04861. Preprint. Disponível em: https://arxiv.org/abs/1704.04861. Acesso em: 26 out. 2026. 27, 28

JACOB, B. *et al.* Quantization and training of neural networks for efficient integer-arithmetic-only inference. *In*: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2018. p. 2704–2713. Disponível em: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html. Acesso em: 26 out. 2026. 28

JOCHER, G. *et al. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. 2026. ArXiv:2606.03748v1. Preprint. Submetido em 2 jun. 2026. Disponível em: https://arxiv.org/html/2606.03748v1. Acesso em: 26 out. 2026. 26

KÜPPERS, F. *et al.* Multivariate confidence calibration for object detection. *In*: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, 2020. Disponível em: https://arxiv.org/abs/2004.13546. Acesso em: 26 out. 2026. 24

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, p. 436–444, 2015. Disponível em: https://doi.org/10.1038/nature14539. Acesso em: 26 out. 2026. 21

LECUN, Y. *et al.* Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998. Disponível em: https://bottou.org/papers/lecun-98h. Acesso em: 26 out. 2026. 22

ONNX. *ONNX Concepts*. s.d. Documentação técnica. Disponível em: https://onnx.ai/onnx/intro/concepts.html. Acesso em: 26 out. 2026. 26, 27, 28

OpenCV. *Geometric Image Transformations*. s.d. Documentação técnica, versão 4.13.0. Disponível em: https://docs.opencv.org/4.13.0/da/d54/group__imgproc__transform.html. Acesso em: 26 out. 2026. 20

PASCAL VOC. *The PASCAL Visual Object Classes Challenge 2010*. 2010. Disponível em: https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2010/. Acesso em: 26 out. 2026. 20

Raspberry Pi Ltd. *Raspberry Pi Zero 2 W*. [S.l.], 2024. Product brief. Disponível em: https://datasheets.raspberrypi.com/rpizero2/raspberry-pi-zero-2-w-product-brief.pdf. Acesso em: 26 out. 2026. 14, 29

REDMON, J. *et al.* You only look once: Unified, real-time object detection. *In*: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016. p. 779–788. Disponível em: https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html. Acesso em: 26 out. 2026. 14, 25

REZATOFIGHI, H. *et al.* Generalized intersection over union: A metric and a loss for bounding box regression. *In*: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019. p. 658–666. Disponível em: https://openaccess.thecvf.com/content_CVPR_2019/html/Rezatofighi_Generalized_Intersection_Over_Union_A_Metric_and_a_Loss_for_CVPR_2019_paper.html. Acesso em: 26 out. 2026. 25

Stanford University. *CS231n: Convolutional Neural Networks*. s.d.a. Notas de curso. Disponível em: https://cs231n.github.io/convolutional-networks/. Acesso em: 26 out. 2026. 22, 23

Stanford University. *CS231n: Image Classification*. s.d.b. Notas de curso. Disponível em: https://cs231n.github.io/classification/. Acesso em: 26 out. 2026. 19, 20

Stanford University. *CS231n: Neural Networks Part 1*. s.d.c. Notas de curso. Disponível em: https://cs231n.github.io/neural-networks-1/. Acesso em: 26 out. 2026. 21

Stanford University. *CS231n: Neural Networks Part 2*. s.d.d. Notas de curso. Disponível em: https://cs231n.github.io/neural-networks-2/. Acesso em: 26 out. 2026. 21

SZE, V. *et al.* Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, v. 105, n. 12, p. 2295–2329, 2017. Disponível em: https://arxiv.org/abs/1703.09039. Acesso em: 26 out. 2026. 26, 27, 29

Tencent. *ncnn*. s.d. Repositório e documentação oficial. Disponível em: https://github.com/Tencent/ncnn. Acesso em: 26 out. 2026. 26, 28

Ultralytics. *ultralytics/utils/nms.py*. 2026. Código-fonte, versão 8.4.108. Disponível em: https://raw.githubusercontent.com/ultralytics/ultralytics/v8.4.108/ultralytics/utils/nms.py. Acesso em: 26 out. 2026. 24, 25

Ultralytics. *Model Prediction with Ultralytics YOLO*. s.d.a. Documentação técnica. Disponível em: https://docs.ultralytics.com/modes/predict/. Acesso em: 26 out. 2026. 19, 20

Ultralytics. *Object Detection with Ultralytics YOLO*. s.d.b. Documentação técnica. Disponível em: https://docs.ultralytics.com/tasks/detect/. Acesso em: 26 out. 2026. 24

Ultralytics. *Ultralytics YOLO26*. s.d.c. Documentação técnica. Disponível em: https://docs.ultralytics.com/models/yolo26/. Acesso em: 26 out. 2026. 14, 26

ZHANG, A. *et al. Dive into Deep Learning*. Cambridge: Cambridge University Press, 2023. Disponível em: https://d2l.ai/. Acesso em: 26 out. 2026. 23

APÊNDICES

APÊNDICE A – TÍTULO

APÊNDICE B – TÍTULO

ANEXOS

ANEXO A – TÍTULO

ANEXO B – TÍTULO